

GraphMine

From Business Questions to Auditable GPU Graph Analysis

An intuition-first account of the architecture, training,
evaluation, and feedback workflow

Project technical note

Implementation and experiment snapshot: October 2026

Includes records from 1–4 October 2026 UTC

Abstract

GraphMine lets a user ask practical questions about connected records without having to name a graph algorithm or a GPU implementation. A language model interprets the question, a deterministic application checks the proposed analysis, a precompiled C++/CUDA library computes the result, and the application turns that result into named records, tables, explanations, and visualizations. This note explains that design from intuition to implementation. It also describes the two completed local routing-adapter training runs and the business question curriculum used by the current pilot. On 124 authored business routing cases, the matched NF4 base passed 113 and the business adapter passed 118 under the recorded application-aware review. This is limited routing evidence: the review was refined during development, a grouping regression remains, and a general improvement in complete user workflows has not been established. The paper therefore separates working software, completed training, measured results, and proposed next steps. The installed pilot is currently stopped at the user’s request; its weights and saved data are preserved.

How to read this note. Part I develops the idea with an ordinary staffing question. Part II follows that question through the software. Part III explains what was trained, how it was tested, and what the numbers mean. Part IV describes the user-testing loop and the work still needed. The appendices identify the source files, experiment records, and commands needed to revisit the system.

Contents

I	The idea, before the machinery	4
1	The problem GraphMine is trying to solve	4
2	A running example: finding teams	4
3	The central design: interpretation, computation, and evidence	5
II	How the application works	5
4	Architecture and the boundary around execution	5
4.1	Who owns each decision?	7
5	From uploaded records to an answer	7
5.1	Domain and data ingestion	7
5.2	Bounded context and routing	8
5.3	Filters and follow-up scope	8
5.4	Planning and the execution gate	8
5.5	Grounding, names, and visualization	9
6	What the graph toolkit currently covers	9
III	What was trained, and what the evidence says	10
7	The learning target	10
8	How the training data was constructed	11
8.1	Small graphs with independently checkable answers	11
8.2	The change to practical business questions	11
9	Fine-tuning mechanics	12
9.1	A small learned update to a frozen base	12
9.2	Learning the response, not copying the prompt	12
9.3	Recorded configuration and completed runs	13
10	Evaluating the system at the right level	14
10.1	Why the matched base matters	14
10.2	What the original candidate showed	14
10.3	What the business candidate showed	15
10.4	A concrete regression and what the validator did	15
10.5	What was complete when work stopped	16
IV	User testing, limitations, and the next learning loop	16
11	Making behavior observable	16
12	Feedback as a review dataset	16
13	What still limits the system	17
13.1	Semantic coverage and model generalization	17
13.2	Evaluation and data limitations	17
13.3	Performance and operations	18
13.4	Access and persistence	18
14	A concrete next research phase	18

A	Reproduction and evidence map	20
A.1	Implementation entry points	20
A.2	Saved training artifacts	20
B	Restarting the existing pilot	21
C	A compact glossary	21

Part I

The idea, before the machinery

1 The problem GraphMine is trying to solve

A project manager might ask, “Which existing groups could work together on a new project?” An investigator might ask for mutually connected accounts to put in a review queue. A researcher might want groups of associated proteins for follow-up experiments. Each question combines a practical goal with a structural question about relationships.

The user usually knows what a record means: an employee, an account, a device, a product, or a protein. They may also know what an edge means: collaboration, payment, communication, purchase, or association. They should not have to know which implementation accepts those records, which parameters are semantic, or which GPU backend was compiled.

GraphMine supplies the bridge between those two levels. It asks for a meaningful business choice when the goal is unclear, translates a sufficiently precise request into a supported computation, runs that computation, and explains the returned evidence using the original records. For example, a good clarification to “Who should I focus on?” asks whether the user wants people who connect otherwise separate teams or people who have worked with many colleagues. That distinction is useful to the user even without naming a centrality measure.

The current system is a graph-analysis application over uploaded data. Its persistent store keeps conversations, files, jobs, results, and review evidence. It does not establish that every business objective can be answered from graph structure. A payment circle is a pattern in the records; deciding whether it is fraud requires evidence outside that pattern.

2 A running example: finding teams

Imagine a small collaboration dataset with five employees: Amina, Ben, Chen, Dina, and Eli. Amina, Ben, Chen, and Dina have all worked with one another. Eli has worked with Amina and Ben, but not with Chen or Dina. This is an illustrative example for explaining semantics, not an additional measured experiment.

Suppose the user asks:

Help me staff projects with existing teams. Show every team of at least three coworkers who have all worked with one another. Leave out a team if it is entirely included in a larger qualifying team, and give me the members’ names.

There are two requested groups: $\{\text{Amina, Ben, Chen, Dina}\}$ and $\{\text{Amina, Ben, Eli}\}$. The smaller triplets inside the four-person group are not separate answers because the request explicitly excludes them.

Intuition. The important work is preserving what “every team” means. Asking for all complete groups, the largest complete group, or every three-person group can produce different correct answers from exactly the same data.

Technical view. Represent the collaboration data as an undirected graph $G = (V, E)$. A subset $C \subseteq V$ is a clique if every distinct pair in C is an edge. The request asks for all inclusion-maximal cliques with $|C| \geq 3$.

Formally, the output family is

$$\mathcal{C}_{\text{maximal}} = \left\{ C \subseteq V : |C| \geq 3, \binom{C}{2} \subseteq E, \nexists v \in V \setminus C \text{ with } \binom{C \cup \{v\}}{2} \subseteq E \right\}. \quad (1)$$

Here $\binom{C}{2}$ means the set of unordered pairs of members of C . “Maximal” means a group cannot be extended; “maximum” means a group has the largest possible size. If the user instead requests every three-person group, including groups inside larger ones, the four-person clique contributes $\binom{4}{3} = 4$ triplets and Eli’s group contributes one more. Returning only the two full groups would answer a different question.

This distinction is also a real limitation of the pilot: a business-adapter route confused exact-size payment groups with full groups in one saved comparison. Section 10.4 explains how that mistake propagated.

3 The central design: interpretation, computation, and evidence

Intuition. Think of the system as an analyst using a tested analysis toolkit. The analyst identifies the question and explains the answer. The toolkit performs the calculation. A supervisor checks that the requested tool and its inputs are allowed and that the answer can be traced back to the calculation.

In GraphMine, the language model fills the interpretation role. The native library supplies graph algorithms. The application performs validation, orchestration, record joins, storage, and presentation. These components have different responsibilities and fail in different ways:

- A router can choose the wrong mathematical problem even when its JSON is valid.
- A planner can choose an invalid parameter or overlook a required input.
- A native implementation can be incorrect or unsuitable for an input; the project therefore exposes implementations that passed its recorded validation.
- A correct numerical result can still be presented badly, with missing names, an unreadable chart, or a claim stronger than the evidence.

This separation makes diagnosis possible. Improving the interface, fixing a filter, and fine-tuning a router are three different changes. A better screen does not demonstrate better model weights. A lower training loss does not show that the user received the correct teams.

Part II

How the application works

4 Architecture and the boundary around execution

The installed pilot has a browser application, a FastAPI service, two model services, a serial native-job worker, and persistent storage. Figure 1 shows their main relationships. The browser can run on another computer on the private network; the application and model processes run on the GPU server.

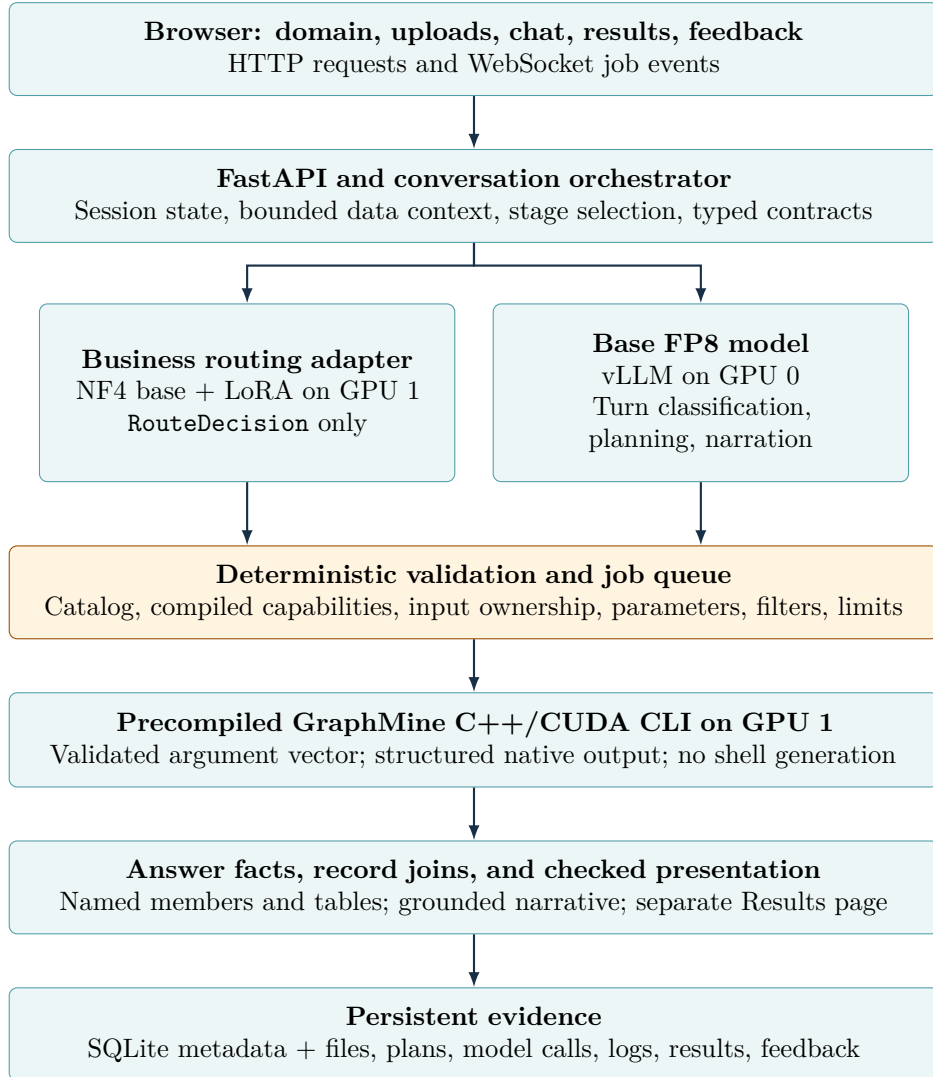


Figure 1: The pilot’s component architecture. The FP8 service is reused after execution for narration; storage records events throughout the request, not only at the final arrow. GPU 1 is shared by the router and native computations.

The current application endpoint uses port 18000. The base model listens on loopback port 8001 and the routing adapter on loopback port 8002. The pilot uses two RTX 6000 Ada GPUs with approximately 48 GiB each. Stable GPU UUIDs select the devices. The FP8 model occupies GPU 0; the NF4 router and native graph work share GPU 1. This is a capacity constraint: two physical GPUs do not imply an otherwise empty GPU for graph processing in the adapter pilot.

4.1 Who owns each decision?

Stage	Current component	Responsibility
Turn classification	Base FP8 model	Decide whether a follow-up needs analysis, explanation, or clarification.
Analysis routing	Business NF4 adapter	Express the objective, filters, requirements, ambiguity, and selected analysis.
Detailed planning	Base FP8 model	Propose settings within one selected operation’s executable contract.
Validation and backend choice	Python application	Check compatibility and select an allowed compiled implementation.
Computation	C++/CUDA library	Produce numerical and structural results.
Facts and presentation	Python plus FP8 narration	Join original records, materialize evidence, and explain selected facts.
Feedback commands	Python application	Save annotations and exports without invoking a model.

Table 1: Only the routing row is changed by the trained business adapter.

The application uses typed objects such as `RouteDecision`, `PlanDraft`, and `ExecutionPlan`. A model response is a proposal to the application. The application constructs the executable identity and command arguments only after validation. Native execution uses `asyncio.create_subprocess_exec`, so generated text is not interpreted as a shell program. This reduces an execution risk; it does not prove that the model understood the user’s intent. [3]

5 From uploaded records to an answer

5.1 Domain and data ingestion

A session starts with one of nine profiles: general graph analysis, fraud, bioinformatics, cybersecurity, social networks, recommendation/e-commerce, knowledge graphs, communications/infrastructure, or transportation/mobility. The profile supplies vocabulary and interpretation context. It cannot change the formal meaning of an operation or make an unsupported one executable.

The system accepts canonical graph JSON, attributed relationship CSV, and text edge lists. Ingestion retains the original upload, constructs a normalized graph, records metadata and hashes, and preserves names and attributes for later joins. Timestamp units and column mappings can be explicit. Auxiliary files have defined roles, such as a pattern, update sequence, or partition. Where typed records determine a routine input unambiguously, the application can derive it rather than asking a novice for a technical file.

Intuition. A node identifier is useful to a computer, but “employee 17” may be useless to the manager. Keeping the connection between internal identifiers and source names is part of

producing the answer, not an optional decoration.

5.2 Bounded context and routing

The model receives a bounded description of the data: entity types, relationship meaning, available fields, metadata, active constraints, and the routing catalog. It does not need to read the entire graph token by token to count a pattern. When the existing description is insufficient, a route can request at most two read-only inspections. Each inspection selects fields and filters and samples at most 20 rows, with relevant counts or statistics supplied by the application.

The routing input builder is shared by runtime and training export. Among its fields are `message`, `graph_context`, `graph_metadata`, `active_constraints`, `pending_request`, `routing_context`, and the description of application preprocessing. This shared contract reduces a common training mismatch: teaching the model on inputs different from those it will receive in the app.

The route contains the formal problem and operation IDs, support status, confidence, unresolved questions, missing information, and an `ApplicationIntent`. The intent explicitly records filters, direction, weight requirements, and temporal semantics. A numerical confidence field is a model output, not a calibrated guarantee of correctness. A route may also request up to three additional independent analyses on the same scope. This is bounded composition; arbitrary dependent analysis pipelines are outside that contract.

5.3 Filters and follow-up scope

Intuition. “Consider only high-value payments” selects records. “Use payment costs when finding the cheapest route” changes the calculation. These requests must not be treated as equivalent.

Filters target vertices or edges and specify an exact field, operator, and value. The available operators are equality, inequality, ordered comparisons, and membership. The application applies them to a recorded graph projection while retaining the original input. A filter on `attributes.score` is distinct from one on `attributes.ascore`; a similar field name is not permission to substitute one for the other.

Let F_V and F_E denote the active vertex and edge predicates. The intended projection can be described as

$$V' = \{v \in V : F_V(v)\}, \quad (2)$$

$$E' = \{(u, v) \in E : u, v \in V', F_E(u, v)\}. \quad (3)$$

The selected algorithm then acts on $G' = (V', E')$. Selecting no matching records can be a valid empty answer. It should not cause the model to invent a different scope or ask the user to relax a precise condition.

Follow-ups use an explicit filter mode. `inherit` retains earlier filters and adds or updates constraints; the merge key is target, field, and operator. `replace` supplies a complete new filter set, and `clear` removes the scope restrictions. Interpretation-only follow-ups can reuse an existing result. A changed computation must return to planning.

5.4 Planning and the execution gate

After routing, the detailed planner sees the selected operation’s actual contract. Its `PlanDraft` has one of three states: `ready`, `needs_information`, or `unsupported`. Only a ready draft may

contain a plan. For a multi-analysis request, the application validates the planned steps before queueing them.

The validator checks the problem/operation relationship, backend compatibility, parameter names and values, requested optional outputs, required auxiliary inputs, session ownership, paths, and semantic capability limits. The compiled binary is probed to determine what is actually available on this host. A catalog entry alone does not make an optional backend installed.

When a backend is set to `auto`, a deterministic selector can use a saved, correctness-gated benchmark policy, graph feature buckets, compiled availability, and compatibility checks. The model is not trained here to synthesize CUDA code or learn the fastest kernel. Backend selection and language routing are separate mechanisms.

The worker serializes native jobs on the configured compute GPU, records the argument vector, captures stdout and stderr, and enforces cancellation and time limits. Durable job states distinguish queued, running, interpreting, completed, failed, and cancelled work. After a server restart, queued work can resume; interrupted running or interpreting jobs are marked failed with `server_restarted` rather than silently counted as completed.

5.5 Grounding, names, and visualization

Native output is joined back to the input records before narration. The answer builder materializes such items as group membership, source names, selected entities, metrics, field-specific statistics, and limitations. It checks that referenced vertices belong to the analysis input. The analyst chooses from recorded facts and supported views; its implications remain distinct from computed findings.

The browser separates Chat from a full-width Results page. Results can contain networks, member tables, distributions, or other supported views appropriate to the operation. Saved analyses and stable result URLs let a user revisit a previous computation. An offline HTML report preserves an interactive view without requiring a running model server. The plotted coordinates are a visual layout, not extra evidence about the underlying business phenomenon.

6 What the graph toolkit currently covers

The catalog contains 20 formal problem definitions. Twelve have validated library support, represented by 13 runnable operations and 26 cataloged validated backends. Triangle counting and dynamic triangle counting are separate operations under one problem definition. The earlier V1 host acceptance recorded 23 compiled backends; three optional Kokkos community implementations were disabled in that build. These counts describe the recorded portfolio, not universal correctness over all possible graphs. [4, 5]

Eight definitions remain explicitly unavailable: densest subgraph, densest k -subgraph, k -truss decomposition, graphlet counting, frequent subgraph mining, overlapping community detection, link prediction, and graph anomaly detection. The application should explain that boundary rather than substitute a nearby supported calculation.

There are also limits within supported families. The current application gate does not permit weights to act as costs or strengths in native computation. It permits filtering by recorded attributes. Direction-preserving requirements are restricted by the available operation contract. The temporal path currently accepts the ordered pattern $A \rightarrow B$, $B \rightarrow C$, $A \rightarrow C$ on three distinct entities; it does not implement arbitrary event chains. The server converts the requested time window into the dataset's known units once. These distinctions are checked outside the

Operation(s)	Kind of question represented
Maximal cliques; maximum clique; k -cliques Quasi-cliques; k -core	All full mutually connected groups; a largest such group; or fixed-size groups. Dense groups with the operation’s defined density condition; or a surviving subgraph with a specified minimum internal degree.
Maximal bicliques	Complete relationships between two types of entity, such as customers and products.
Triangle counting; dynamic triangle counting	Three-way patterns in a graph, including the supported update-based operation.
Graph motifs; subgraph isomorphism	Supported small structural patterns and matches to a supplied pattern.
Temporal motif mining	The supported ordered three-entity event pattern within a time window.
Community detection; betweenness centrality	A connection-based partition; or intermediary positions on unweighted shortest paths.

Table 2: Capability families. Exact inputs, outputs, and mathematical promises come from the selected catalog contract. A heuristic community partition does not promise that every pair inside a community is connected.

model as well as described in its prompt.

Part III

What was trained, and what the evidence says

7 The learning target

Intuition. The model already has broad language ability. The local training tries to teach it this application’s vocabulary of decisions: which supported analysis answers a request, which constraints matter, when to ask a useful question, and when the available toolkit cannot provide the requested answer.

The completed runs train a **RouteDecision** adapter. Their targets are structured routing responses, not final numerical answers, GPU programs, detailed execution plans, or private reasoning traces. The current pilot applies the business adapter only to routing. Turn classification, detailed planning, and answer narration continue to use the existing FP8 model. [6, 7]

The recorded base identifier is **Qwen/Qwen3.8-27B**, loaded from a pinned local checkpoint. The separate serving identifier is **Qwen/Qwen3.8-27B-FP8**. These are the identifiers in this project’s configuration and experiment evidence. Reproduction requires the recorded checkpoint revision and file hashes; a matching name alone is insufficient.

For one example, the input includes the system routing prompt, data context, catalog context, and a practical question. The assistant target says, in typed form, which problem is intended and what requirements apply. A simplified illustration of part of a staffing target is:

```
{
  "problem_id": "maximal_clique_enumeration",
  "operation_id": "maximal-cliques",
  "supported": true,
```

```

"ambiguity": [],
"intent": {
  "objective": "Find existing teams whose members all worked together",
  "filters": [],
  "requires_edge_weights": false,
  "requires_direction": false
}
}

```

This is an explanatory excerpt, not a complete executable response. Exact size thresholds and optional output settings are subsequently handled by the detailed planner within the selected operation contract. Natural-language objectives remain important, so merely matching an operation ID does not establish a complete correct analysis.

8 How the training data was constructed

8.1 Small graphs with independently checkable answers

The source curriculum contains 48 distinct synthetic graphs and 396 task cards: 288 executable native computations and 108 clarification/refusal contracts. It uses a recorded seed, 271828. The associated native audit passed 396/396 checks. This total includes the non-execution contracts; it does not mean 396 GPU computations were run.

Small graphs make transparent reference calculations possible. The independent Python oracles deliberately use simple enumeration and other direct checks, with limits of 12 vertices and 24 events. For cliques, for example, the oracle can enumerate subsets and inspect all required pairs. The native output is checked against that reference before a task becomes a training candidate. The oracle code is separate from the production graph projection and native adapters. Its bounded test scope is not a scalability benchmark.

The original adapter exported 192 training rows and 72 validation rows; 132 test-family cases were excluded from both weight updates and validation loss. Graph-family separation is useful, but the original wording was heavily reused: 130 of the 132 test question strings also appeared in training. A model could therefore benefit from familiar language templates even on different graphs. That fact limits what the original perfect routing score can establish.

8.2 The change to practical business questions

The user clarified that training requests should sound like application needs, without asking people to name the underlying GPU program. A separate curriculum therefore rewrote appropriate task cards around staffing, payment review, service planning, retail campaigns, incident review, and laboratory follow-ups. Broad requests such as “Which accounts should we investigate first?” train domain-language clarification rather than a guessed analysis.

The new curriculum retained a source card’s graph, computation, filters, and requirements when reusing its saved native audit. The builder rechecked those contracts and outputs, omitted 48 source cards, and added 24 broad-goal cards, giving 372 cards in total. This revalidation reused evidence; it did not rerun 372 native jobs. [8]

Question banks have no exact string overlap across these three splits, and graph families remain separated. Nevertheless, the questions and labels come from the same authoring process and related task types. Repeated wording within a split also means 180 training cards are not 180

Split	Task cards	Distinct question strings	Used for
Training	180	61	Weight updates
Validation	68	59	Token-loss evaluation
Evaluation	124	61	Generation-based comparison

Table 3: Business-question curriculum. A card combines a question, graph, context, and target; it is not an independent customer request.

independently phrased questions. The business wording has not received independent language review. Checking a graph answer cannot by itself prove that a sentence communicates the intended task to an unfamiliar person.

An earlier, separate teacher/reviewer pilot generated 10 reviewed routing candidate rows and exercised the data pipeline. Those records are not the data lineage of the two completed adapters discussed here. The completed runs use the oracle-gated synthetic exports identified above. They should not be described as distillation of a stronger model’s hidden reasoning. Public workplace and protein data were used in development evaluation, while the reserved OpenFlights source remained unused by the final evaluation path at handoff. [9]

9 Fine-tuning mechanics

9.1 A small learned update to a frozen base

Intuition. Instead of rewriting the entire model, keep its existing weights fixed and learn a smaller set of adjustments. Store the large fixed component compactly so the training run fits on one available GPU.

For a weight matrix $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, LoRA represents the learned update with two smaller matrices:

$$W_{\text{effective}} = W_0 + \frac{\alpha}{r}BA, \quad A \in \mathbb{R}^{r \times d_{\text{in}}}, \quad B \in \mathbb{R}^{d_{\text{out}} \times r}. \quad (4)$$

Only A and B are trained. The low rank r restricts the number of adjustable parameters. This is the LoRA mechanism. [1]

The implementation uses a QLoRA-style configuration: the frozen base is loaded in 4-bit NormalFloat (NF4), with double quantization, while matrix computation uses BF16. NF4 reduces storage for the base, and double quantization compresses the quantization constants. These choices allow gradients to update adapters through a quantized frozen model. [2] The project uses ordinary `adamw_torch`; it does not claim to reproduce every optimizer choice or benchmark result from the QLoRA paper.

LoRA targets eligible linear modules through PEFT’s `all-linear` selection. The saved run reports 116,727,808 trainable parameters out of 27,012,726,272 total parameters, approximately 0.432%. There are 992 saved adapter tensors, with 496 LoRA-B tensors. Embedding and output matrices remain frozen; the training implementation explicitly keeps those large vocabulary matrices in BF16 to avoid an unnecessary FP32 memory cost.

9.2 Learning the response, not copying the prompt

Each example is serialized with the model’s chat template. The system and user tokens are context, while the assistant response tokens are the supervised target. The implementation

gives context labels the ignore value -100 and checks that the full conversation begins with the exact expected prompt prefix. It rejects an example that exceeds the configured token limit instead of silently truncating it.

If x_i is the prompt, y_i is the target route, and ϕ denotes the adapter parameters, the intended assistant-token objective is

$$\mathcal{L}(\phi) = -\frac{1}{\sum_i |y_i|} \sum_i \sum_{t=1}^{|y_i|} \log p_{\theta_0, \phi}(y_{i,t} \mid x_i, y_{i,<t}), \quad (5)$$

where θ_0 is frozen. The trainer computes masked next-token cross-entropy on assistant positions in each microbatch and accumulates gradients before an optimizer step. The equation expresses the supervision target; exact averaging follows the implementation’s batch reductions.

Thinking is disabled in the routing chat template. There is no reasoning-trace target in these examples. To reduce memory, the custom trainer requests logits only for the response portion needed by the loss rather than the entire long prompt. Gradient checkpointing trades additional computation for less stored activation memory. None of these changes trains the native graph library.

9.3 Recorded configuration and completed runs

Setting	Recorded value
Base and precision	Pinned local 27B checkpoint; NF4 with double quantization; BF16 computation
LoRA rank / alpha / dropout	16 / 32 / 0.05
Target modules	Eligible linear modules, all-linear ; no trained bias
Epochs	1
Microbatch / gradient accumulation	1 example / 4 microbatches
Nominal effective batch	4 examples on one GPU
Learning rate / schedule	10^{-4} / cosine
Optimizer / gradient clipping	adamw_torch / maximum norm 1.0
Sequence limit	8,192 tokens; longest business example: 7,333
Seed	271828
Memory controls	Gradient checkpointing; response-only logits; frozen vocabulary matrices in BF16
Evidence	Pinned package versions, checkpoint hashes, export provenance, optimizer checkpoints, per-step metrics, saved adapter

Table 4: Settings shared by the completed original and business routing runs. The saved source records the exact scheduler arguments and trainer behavior.

The original run, **finetune-qwen27b-v3**, completed 48 optimizer steps. The business run, **finetune-qwen27b-business-v1**, completed 45. With one epoch and an effective batch of four, these counts follow $192/4 = 48$ and $180/4 = 45$. The business run’s recorded training runtime is 3,501.29 seconds, about 58.4 minutes; model loading and verification are not included in that trainer runtime. Its recorded peak allocated GPU memory is about 26.6 GiB. Peak allocator use is not the same measurement as total process memory reported by a GPU monitor.

Each final adapter occupies 467,047,680 bytes. Verification found all 992 tensors finite and every LoRA-B tensor nonzero. All 496 business LoRA-B tensors differ from the first candidate. This establishes that real updates were saved; it does not establish a useful behavioral improvement. The business run is a separate candidate against the same pinned base and settings, not a continuation that silently overwrote the original adapter.

Candidate	Train / validation	Steps	Mean train loss	Validation loss
Original routing	192 / 72	48	0.125474	0.000754
Business routing	180 / 68	45	0.142746	0.014998

Table 5: Completed training measurements. These losses are not task accuracy; the two candidates also use different data distributions.

10 Evaluating the system at the right level

Intuition. A routing test asks whether the analyst chose the right calculation. A workflow test asks whether the user actually got a useful, correct result. Passing the first test is helpful but cannot substitute for the second.

The project uses several evidence layers:

1. **Native correctness checks:** compare computations against bounded independent references and documented algorithm-specific checks.
2. **Application tests:** check validation, storage, ownership, filters, recovery, commands, and presentation behavior.
3. **Static routing comparisons:** generate structured decisions for fixed question/context pairs and score their semantics and effective behavior.
4. **Full workflow replays:** run routing, planning, native computation, answer construction, and follow-ups in the application.
5. **User trials:** assess whether actual users can ask useful questions and understand the results. The deployment acceptance is not such a study.

10.1 Why the matched base matters

The controlled local comparison uses the same frozen checkpoint, NF4 loading, tokenizer, prompt, response schema, and decoding settings in both arms. The adapter is enabled in one arm and disabled in the other. Generation is greedy and constrained with XGrammar to the JSON response schema. Schema constraints help produce parseable output but cannot force the correct analysis choice.

The existing FP8 serving model is reported separately. It uses a different precision and serving engine. Comparing its score directly with the NF4 adapter mixes the effect of the learned update with those runtime differences. Similarly, amortized batch generation time is not end-user latency for an entire analysis.

10.2 What the original candidate showed

On the 132 original synthetic routing cases, the recorded application-aware scores were 123/132 for the matched NF4 base and 132/132 for the original adapter. The original strict scores were 110/132 and 132/132. The wording overlap described earlier limits this result to a narrow development finding. [10]

The harder 53-case challenge also illustrates why label review matters. An initial application review gave 46/53 for the base and 52/53 for the adapter. A subsequent audit of production behavior and missing-input labels produced 49/53 for both, with three improvements and three

regressions in paired cases. All these versions are preserved; choosing only the most favorable version would misrepresent the evidence. [11]

Full synthetic application comparisons passed 20/21 turns in both local arms. On a later real-data development comparison, the original grades were 22/22 for the base and 20/22 for the original adapter. A separate audit found that both adapter flags compared numeric 0.0 with 0 on otherwise identical equality filters, with no other reported issue. The audit preserves the original grades and does not replace them with a new aggregate claim. These findings do not establish an end-to-end quality gain. [12, 13]

10.3 What the business candidate showed

Routing arm	Strict score	Application review	Reviewed rate
Matched NF4 base	99/124	113/124	91.1%
NF4 base + business adapter	118/124	118/124	95.2%
Separate FP8 reference	102/124	115/124	92.7%

Table 6: Business routing development results. Only the first two rows form the matched adapter comparison. The review was refined after inspecting development outputs; this is not an untouched final benchmark.

The net reviewed difference is five of 124 cases, or approximately 4.0 percentage points. In paired terms, 112 cases passed in both arms, six improved, five failed in both, and one regressed. Five improvements clean up speculative routing metadata in cases already asking for clarification. One removes an unnecessary clarification about a protein-panel starting pool. They are not six newly successful complete computations. [14, 15]

The application-aware review accounts for how the production route blocker actually treats a response, permits bounded inspections, and disregards temporal fields irrelevant to non-temporal operations. Original outputs and strict scores remain available. All 124 outputs in each business arm were schema-valid, yet semantic errors remained. The scorer also recorded two execution proposals against clarification labels in each local arm; no native computations were launched by this static comparison. A syntactic success rate should therefore not be reported as a safety or task success rate.

The six business-adapter failures include four payment-group operation errors across two wording patterns and two graphs, plus two cases where the authored label expected clarification for a service-team goal. Those labels are themselves part of an authored development protocol, not an independent user judgment. Related templates, repeated graphs, one seed, and one comparison pass prevent a broad statistical generalization from these totals.

10.4 A concrete regression and what the validator did

One saved request asked for every three-account mutual-payment set, explicitly including sets contained in larger circles. The business adapter selected the full-group operation. The matched base selected the fixed-size operation.

A diagnostic replay reused those saved routes and ran fresh FP8 planning, native work where permitted, and answer generation. The base route produced all five requested triplets. The business route led the planner to an unsupported response and no computation. The planner prevented a mismatched calculation, but it did not recover the intended operation. [16]

This is an example of a useful execution boundary with an incomplete user experience. The

system avoided presenting the wrong grouping result, but the user still did not get an answer to a supported request. The replay used saved routes, so it is a failure diagnosis, not a fresh routing accuracy estimate or a controlled latency experiment.

10.5 What was complete when work stopped

Both training runs are complete. The business routing comparison is complete. The separate serving FP8 reference passed 21/21 turns across nine practical application conversations, but the corresponding matched business-adapter workflow comparison had not started when the user paused evaluation.

The original adapter’s run on the new business questions stopped with 46 of 124 planned calls saved; no complete score is reported for it. The business candidate’s 53-case retention check and 22-turn real-data comparison had not started. The final reserved source remained unused. Historical files that say “not deployed” describe their creation time; a later milestone installed the business adapter in the pilot. [17, 18]

The pilot deployment recorded 295 passing Python tests, two browser scenarios, lint and JavaScript syntax checks, and a live model/native/browser acceptance. That acceptance returned two expected named collaboration groups and verified the use of the business router followed by FP8 planning and narration. It was marked as assistant evaluation on authored data, not real-user feedback. It verifies that the integration worked for that case; it does not close the unfinished model-quality gates.

Part IV

User testing, limitations, and the next learning loop

11 Making behavior observable

The chat sidebar shows observable activity such as planning, running a job, and interpreting a result. It also offers an expandable view of reasoning text actually returned by the local Qwen model for stages that emit it. That text is displayed after the relevant response is available; it is not a live transcript of every internal operation.

The trained routing stage uses thinking-disabled generation, so the application does not invent a reasoning trace for it. For other stages, emitted model reasoning is an explanation artifact, not proof that its statements or causal account are correct. Native results, validated plans, source records, and logs remain the evidence for what was computed. The deployment check verified an exact match between one displayed 2,282-character planning response and the saved model output; it did not verify the truth of every sentence in that text.

12 Feedback as a review dataset

Intuition. When a user says an answer is wrong, the useful training unit is more than that sentence. It includes what they asked, which data and model version were involved, what the system did, and what they expected instead.

The app intercepts local commands before model dispatch. Both slash and backslash prefixes work. They are saved in history but excluded from current and future model prompts. Feedback does not automatically edit an earlier result, start a new computation, or update model weights.

Command	What is recorded or returned
<code>/feedback NOTE</code>	A problem or observation about the latest request.
<code>/feedback NOTE EXPECTED</code>	The same annotation with expected behavior; this is a second format of the same command.
<code>/correct ANSWER</code>	A proposed corrected answer for review.
<code>/rate 1-5 [NOTE]</code>	A score with an optional explanation.
<code>/note NOTE</code>	An observation about the testing experience.
<code>/export</code>	The full session evidence archive.
<code>/status</code>	Configured model/version and stage assignments.
<code>/help</code>	Local command help.

Table 7: Commands are typed in the browser chat. The app handles them without calling the language model.

The sidebar’s *Leave feedback* targets the latest chat turn. *Give feedback* below an answer targets that answer, while *Review this answer* on Results targets the displayed result. These buttons provide forms for the same categories: feedback, correction, rating, and note. They make older answers reviewable without attaching a correction to a later turn.

Each annotation retains target links, tester alias where supplied, deployment context, and a snapshot of the conversation at annotation time. Session archives retain broader evidence including uploads, model calls, native outputs, and logs. A JSONL feedback export contains one record per annotation and links to the related session evidence. Records are marked `unreviewed`, with `automatically_used_for_training: false`. [19]

User corrections are valuable proposals, but users can misunderstand a task or make factual mistakes. A future training export should first distinguish a misunderstood request, missing capability, bad computation, poor explanation, presentation defect, and preference. These require different remedies. Repeated corrections must not leak across training and evaluation splits through the same conversation or source dataset.

13 What still limits the system

13.1 Semantic coverage and model generalization

The toolkit supports a defined portfolio, with restrictions on weights, direction, and temporal patterns. A capable language model cannot create a missing implementation by describing one. Within the portfolio, the account grouping regression shows that semantically close operations remain difficult. Training on synthetic cards and related phrasing has not established reliable interpretation of arbitrary business questions.

13.2 Evaluation and data limitations

The evaluation questions, small graphs, and oracles were designed for development. Original wording overlap, later grading changes, and label audits are material limitations. The public-data samples were small and selected for exact checking, not representative of full workplace, biological, or transport datasets. A reserved source can reduce one kind of development leakage, but it cannot prove that the pretrained model has never encountered similar data or tasks.

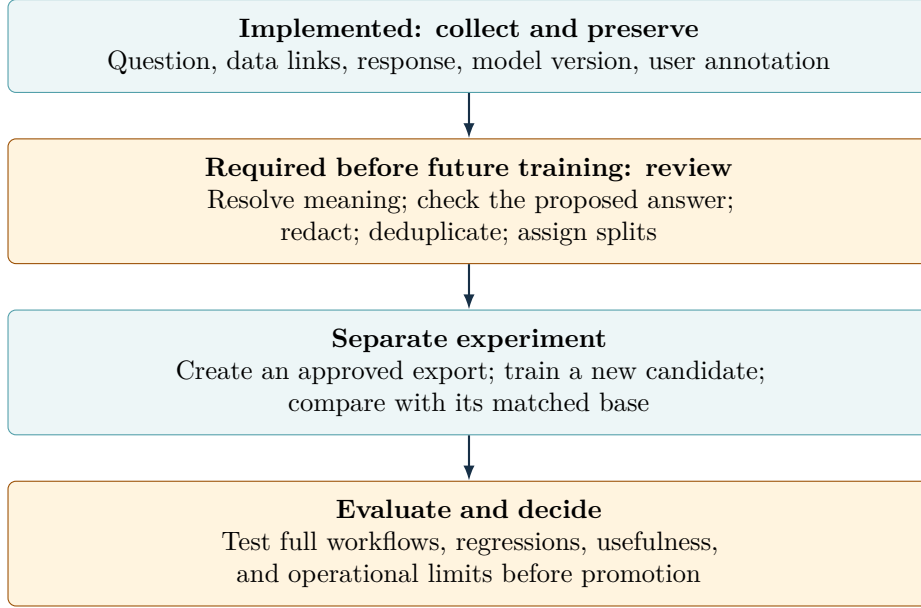


Figure 2: The intended feedback-to-improvement loop. Collection exists now. Automatic training from newly collected pilot annotations is not implemented or enabled by the completed adapter runs.

13.3 Performance and operations

No general GPU speedup, concurrent-user throughput, or large-graph capacity claim is established by the pilot acceptance. Enumerating groups can create a large output even when the graph fits in memory. The browser and model receive bounded views, so truncation and partial materialization must be visible. GPU 1 also holds the router, reducing the memory available for native work. The serial worker limits simultaneous native jobs but is not a complete multi-user resource scheduler.

13.4 Access and persistence

The pilot uses a shared API token and a shared workspace. Request validation checks session/-file consistency, but that does not provide per-user account isolation: people with the shared credential are not separate security tenants. The pilot also saves conversations and uploaded data for review. Expanding to external users requires explicit choices about access, retention, deletion, consent, and which evidence may enter a training dataset.

The installation has since been stopped at the user’s request. API and router automatic startup are disabled, and the FP8 server is stopped. This document describes the installed implementation and recorded acceptance, not a claim that an endpoint is presently running.

14 A concrete next research phase

The next phase should turn observed user difficulties into testable changes. The following are proposed work, not results already achieved:

1. **Repair grouping semantics.** Build contrasting practical requests for full groups, the largest group, and all fixed-size groups; independently check the labels and replay the

known payment failure.

2. **Finish the pending comparisons.** Complete matched business workflows, retention checks, and real-data comparisons before using the untouched reserved source for a declared final protocol.
3. **Collect actual user questions.** Measure clarification usefulness, task completion, and comprehension of names, tables, and visualizations. Keep assistant-authored acceptance feedback separately labeled.
4. **Review before training.** Produce a curated candidate dataset with source provenance and conversation/source-level split boundaries. Keep raw annotations and reviewed targets as distinct artifacts.
5. **Attribute improvements.** Compare model updates against the same base/runtime; evaluate code and UI changes separately where possible. Include multiple runs or seeds and uncertainty estimates appropriate to correlated cases.
6. **Measure deployment limits.** Test realistic graph sizes, result volumes, GPU sharing, and concurrent requests. Add account-level controls before treating the shared pilot as a multi-tenant product.

The research question is whether the complete system helps people obtain the right analysis from their own language and data. The present evidence supports a working integration and a limited routing improvement. Answering that broader question requires the application and user evaluations above.

A Reproduction and evidence map

All paths in this appendix are relative to the repository root. The source snapshot has substantial existing uncommitted work; a Git revision alone would not identify every file described here. The companion `evidence_manifest.json` records SHA-256 hashes of selected source and report files read for this note. It is an index of evidence, not a new run of the experiments. No private API token or user conversation is embedded in the paper.

A.1 Implementation entry points

File or directory	Role
<code>graphmine_catalog.json</code>	Formal problems, support boundary, and backend provenance.
<code>graphmine_manifest.json</code>	Compact operation and CLI contract.
<code>library/</code>	C++/CUDA interfaces, adapters, native CLI, and tests.
<code>agent/graphmine_agent/models.py</code>	Typed routing, planning, result, and feedback contracts.
<code>agent/graphmine_agent/agent_service.py</code>	Conversation stages and orchestration.
<code>agent/graphmine_agent/graph_store.py</code>	Ingestion, data context, inspection, and projections.
<code>agent/graphmine_agent/graph_context.py</code>	Validation and native execution.
<code>agent/graphmine_agent/planning.py</code>	
<code>agent/graphmine_agent/execution.py</code>	
<code>agent/graphmine_agent/selector.py</code>	Deterministic backend selection.
<code>agent/graphmine_agent/answers.py</code>	Facts, source-record joins, tables, and follow-up choices.
<code>agent/graphmine_agent/presentation.py</code>	Pinned business router service.
<code>agent/graphmine_agent/adaptor_server.py</code>	Token masking, adapter training, checkpoints, and status.
<code>agent/graphmine_agent/learning/finetune.py</code>	Business export and independent bounded answer checks.
<code>agent/graphmine_agent/learning/business_curriculum.py</code>	
<code>agent/graphmine_agent/learning/oracles.py</code>	
<code>agent/graphmine_agent/learning/adaptor_eval.py</code>	Matched static routing and conversation runtimes.
<code>agent/graphmine_agent/learning/adaptor_workflows.py</code>	
<code>agent/graphmine_agent/commands.py</code>	Local commands and saved evidence.
<code>agent/graphmine_agent/history.py</code>	
<code>agent/graphmine_agent/static/</code>	Browser chat, results, and feedback UI.

A.2 Saved training artifacts

The source curriculum and its native audit are `.graphmine-learning/finetune-corpus-v2/` and `.graphmine-learning/finetune-native-v1.json`. The practical curriculum and exported supervision are `.graphmine-learning/business-curriculum-v1/` and `.graphmine-learning/business-routing-data-v1/`.

The completed run directories are `.graphmine-learning/finetune-qwen27b-v3/` and `.graphmine-learning/finetune-qwen27b-business-v1/`. Each keeps `run.json`, `status.json`, `metrics.jsonl`, checkpoints, and final adapter files. The business base checkpoint revision is `1d4bf0f2ff6012fd82039f2fa52739d0dd7c60c0`. The business adapter SHA-256 is `dd7017735094300c4a89f217cb094c0d8b9981d1c7d86182596a7a6f7ef5f4d6`. The original adapter SHA-256 is `276cfd92ab692442ead80bb0170af84860d0ce604f89b9bdf1fcacc152449b75`.

Recorded training package versions are PyTorch 2.13.0, Transformers 5.18.0, PEFT 0.21.2, bitsandbytes 0.50.2, Accelerate 1.15.0, Datasets 5.0.1, and flash-linear-attention 0.5.2. The matched evaluation adds XGrammar 0.2.8 and apache-tvm-ffi 0.1.14.post1. These are observed environment pins, not a statement that arbitrary newer versions produce equivalent results.

The routing service verifies the adapter fingerprint, training configuration, base files, runtime prompt, and response schema. The app verifies the router’s identity. If those checks fail, the intended behavior is a visible failure rather than silently using a different routing model. Reproduction should retain that binding between evaluated artifact and deployed artifact.

B Restarting the existing pilot

Run these commands on the GPU server as the existing installation user. They reuse the saved configuration, data, and weights. They do not resume training or the stopped evaluation queue.

In terminal 1:

```
cd /home/wajid/projects/graphDatabaseAgent/graphMine-repo
./deploy/start-vllm.sh
```

Keep the terminal open and wait for model startup to complete. In terminal 2:

```
systemctl --user start graphmine-pilot-router.service graphmine-pilot-api.service
journalctl --user -u graphmine-pilot-router.service -u graphmine-pilot-api.service -f
```

Allow the router’s weights to load. Open <http://127.0.0.1:18000> on the server, or use its private network address from a connected computer. The existing access token is `GRAPHMINE_API_TOKEN` in `agent/.env`; enter it in the UI settings if prompted. Do not regenerate the environment merely to resume the installation. The operator guide is `deploy/README.md`.

To stop the app and router:

```
systemctl --user stop graphmine-pilot-api.service graphmine-pilot-router.service
```

Then press `Ctrl+C` in terminal 1 to stop vLLM. Closing the browser alone does not stop the server processes.

C A compact glossary

Graph

Records represented as vertices and their relationships as edges.

Routing

Selecting the intended analysis and expressing its requirements.

Planning

Filling the selected operation's allowed settings and inputs.

Backend

One native implementation of an operation.

Schema constraint

A restriction on the structure of model output; it does not guarantee the meaning is correct.

Adapter

A saved set of learned parameter updates applied to a frozen base.

Oracle

An independent reference calculation or check for a bounded test.

Holdout

Data reserved from a specified development process. Its validity depends on exactly which questions, graphs, and sources were kept separate.

Grounded fact

A statement materialized from input records, validated plans, or computation outputs, with a traceable source.

Annotation

A saved user observation, correction, rating, or note awaiting review before potential training use.

References

- [1] Edward J. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. 2021. <https://arxiv.org/abs/2106.09685>.
- [2] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. *QLoRA: Efficient Finetuning of Quantized LLMs*. 2023. <https://arxiv.org/abs/2305.14314>.
- [3] GraphMine project. Application implementation and user guide. `agent/graphmine_agent/`; `agent/README.md`. Local working-tree snapshot, October 2026.
- [4] GraphMine project. Formal problem catalog and runtime manifest. `graphmine_catalog.json`; `graphmine_manifest.json`.
- [5] GraphMine project. Recorded V1 release scope and acceptance. `docs/V1.md`. The acceptance snapshot is dated 1 October 2026.
- [6] GraphMine project. Training implementation and saved run configuration. `agent/graphmine_agent/learning/finetune.py`; `.graphmine-learning/finetune-qwen27b-business-v1/run.json`; `.graphmine-learning/finetune-qwen27b-business-v1/status.json`.
- [7] GraphMine project. Completed adapter verification records. `agent/evaluation/reports/finetune-completion-2026-10-03.json`; `agent/evaluation/reports/business-finetune-completion-2026-10-03.json`.
- [8] GraphMine project. Business-question curriculum record. `agent/evaluation/reports/business-question-curriculum-2026-10-03.json`; `.graphmine-learning/business-curriculum-v1/manifest.json`. Distinct-question counts are also recorded in the stop/handoff report.
- [9] GraphMine project. Development and training history. `agent/IMPROVEMENT_PLAN.md`; `agent/evaluation/FINETUNING_PROGRESS.md`. Historical milestones must be read with their dates and later updates.

- [10] GraphMine project. Original 132-case routing comparison and review. `agent/evaluation/reports/adapt-test-review-2026-10-03.json`.
- [11] GraphMine project. Challenge review, including audited label results. `agent/evaluation/reports/adapt-challenge-review-2026-10-03.json`.
- [12] GraphMine project. Original adapter application comparisons. `agent/evaluation/reports/adapt-er-workflow-comparison-2026-10-03.json`; `agent/evaluation/reports/adapt-er-realworld-comparison-2026-10-03.json`.
- [13] GraphMine project. Numeric-filter grading addendum. `agent/evaluation/reports/adapt-er-realworld-numeric-filter-audit-2026-10-03.json`.
- [14] GraphMine project. Business routing comparison and serving reference. `agent/evaluation/reports/adapt-er-business-new-review-v1.json`; `agent/evaluation/reports/adapt-er-business-fp8-review-2026-10-03.json`.
- [15] GraphMine project. Paired business-route change audit. `agent/evaluation/reports/business-routing-change-audit-v1.json`.
- [16] GraphMine project. Saved-route payment-group diagnostic. `agent/evaluation/reports/business-payment-regression-replay-v1.json`.
- [17] GraphMine project. Fine-tuning stop and handoff. `agent/evaluation/reports/finetuning-user-stop-2026-10-03.json`.
- [18] GraphMine project. Pilot integration and acceptance record. `agent/evaluation/reports/pilot-deployment-2026-10-04.json`. Later operational shutdown and restart instructions are in `deploy/README.md`.
- [19] GraphMine project. Local commands, annotation storage, and export implementation. `agent/graphmine_agent/commands.py`; `agent/graphmine_agent/database.py`; `agent/graphmine_agent/history.py`; `agent/graphmine_agent/api.py`.